Ariela Mishaan – Alina Carias – Diego Soto – Ignacio Mendez – Daniel Machic – Marcos Diaz

Taller 3

Enlace al GitHub: https://github.com/ArielaMishaanCohen/Taller_3.git

1. Páginas de documentación

a) MLflow Tracking Quickstart

Se escogió esta página porque introduce las funciones centrales de MLflow Tracking en un recorrido corto y práctico. La guía muestra cómo crear un experimento, entrenar un modelo de scikit-learn y registrar parámetros, métricas y el modelo, ya sea con *autologging* o de forma manual. También explica cómo levantar la interfaz de MLflow para explorar las corridas, y cómo cargar el modelo registrado para hacer predicciones. Esto es relevante porque resuelve un problema habitual al trabajar en equipo: cuando varias personas prueban distintos hiperparámetros y modelos, es fácil perder el registro de qué configuración produjo cada resultado. Con MLflow Tracking cada corrida queda guardada con sus parámetros, métricas, etiquetas y artefactos, lo que permite comparar y reproducir los experimentos de todos.

b) MLflow Scikit-learn Integration

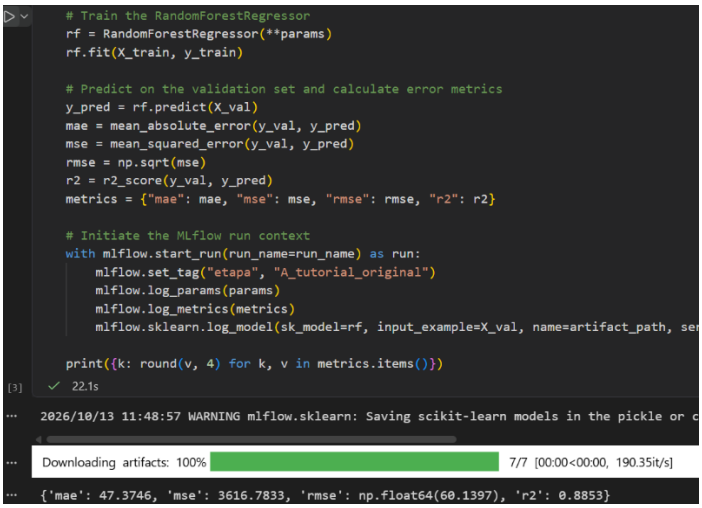
Se seleccionó esta página porque se enfoca en la herramienta de modelado. La página explica que el autologging de MLflow para scikit-learn registra con una sola llamada los parámetros, las métricas de entrenamiento y evaluación, los resultados de validación cruzada y el modelo serializado con su firma (*signature*) y un ejemplo de entrada. También documenta el manejo de GridSearchCV y RandomizedSearchCV donde cada combinación de parámetros se guarda como una corrida hija, y la integración con Optuna para la búsqueda de hiperparámetros. Esto es valioso porque reduce el código de registro que habría que escribir a mano y deja los modelos con la información de entorno y dependencias necesaria para reproducirlos.

c) MLflow Model Registry

Se escogió esta página porque muestra qué ocurre después de entrenar un modelo. El Model Registry es un repositorio centralizado, con API e interfaz gráfica, para administrar un modelo durante todo su ciclo de vida. Sus conceptos principales son: el modelo registrado, las versiones, los alias (por ejemplo, champion) que apuntan a una versión específica, las etiquetas, las anotaciones y el linaje, que enlaza cada versión con la corrida que la generó. Es importante porque marca la diferencia entre experimentar en un notebook y trabajar con prácticas de ingeniería de aprendizaje automático. El versionado permite comparar iteraciones y volver a una versión anterior, y los alias permiten cambiar el modelo en producción sin modificar el código que lo consume.

2. MLFlow en Jupyter Notebook

Se siguió el tutorial para registrar un modelo Random Forest sobre ventas de manzanas, guardando parámetros, métricas (MSE, RMSE, MAE, R^2) y el modelo.



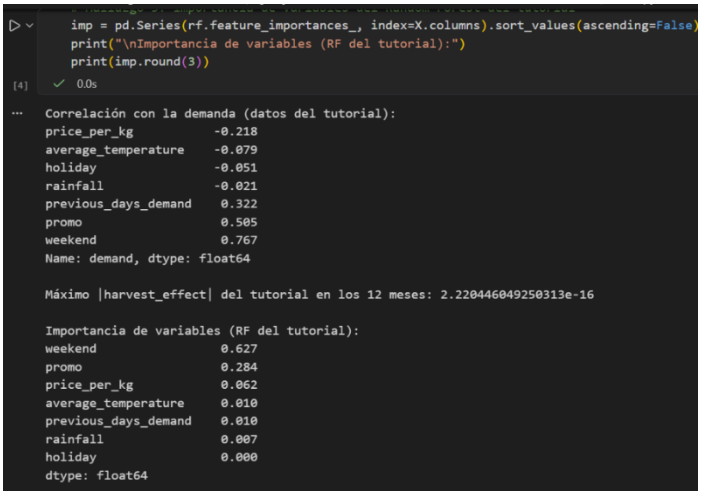
```
# Train the RandomForestRegressor
rf = RandomForestRegressor(**params)
rf.fit(X_train, y_train)

# Predict on the validation set and calculate error metrics
y_pred = rf.predict(X_val)
mae = mean_absolute_error(y_val, y_pred)
mse = mean_squared_error(y_val, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_val, y_pred)
metrics = {"mae": mae, "mse": mse, "rmse": rmse, "r2": r2}

# Initiate the MLflow run context
with mlflow.start_run(run_name=run_name) as run:
    mlflow.set_tag("etapa", "A_tutorial_original")
    mlflow.log_params(params)
    mlflow.log_metrics(metrics)
    mlflow.sklearn.log_model(sk_model=rf, input_example=X_val, name=artifact_path, ser

print({k: round(v, 4) for k, v in metrics.items()})
```

```
[3]  ✓ 22.1s
2026/10/13 11:48:57 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or c
Downloading artifacts: 100% 7/7 [00:00<00:00, 190.35it/s]
{'mae': 47.3746, 'mse': 3616.7833, 'rmse': np.float64(60.1397), 'r2': 0.8853}
```



```
imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
print("\nImportancia de variables (RF del tutorial):")
print(imp.round(3))
```

```
[4]  ✓ 0.0s
Correlación con la demanda (datos del tutorial):
price_per_kg           -0.218
average_temperature    -0.079
holiday                -0.051
rainfall               -0.021
previous_days_demand    0.322
promo                   0.505
weekend                 0.767
Name: demand, dtype: float64

Máximo |harvest_effect| del tutorial en los 12 meses: 2.220446049250313e-16

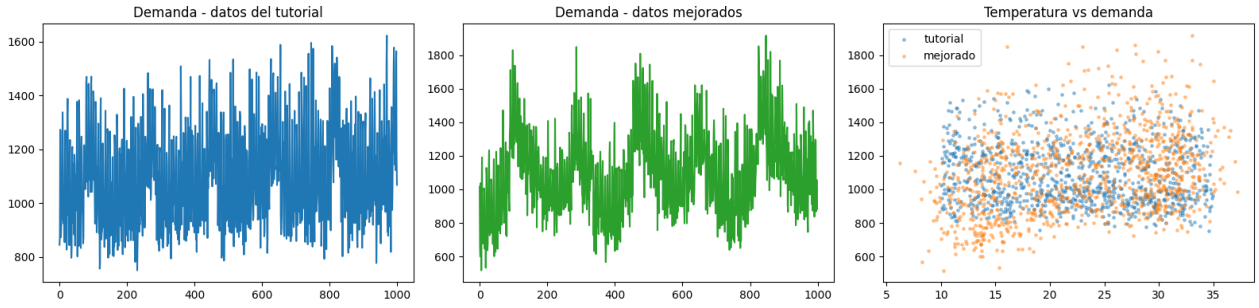
Importancia de variables (RF del tutorial):
weekend                0.627
promo                  0.284
price_per_kg           0.062
average_temperature    0.010
previous_days_demand   0.010
rainfall               0.007
holiday                0.000
dtype: float64
```

El tutorial genera 1,000 días de ventas sintéticas de manzanas con variables como temperatura, lluvia, fin de semana, feriado, precio y promoción, y calcula la demanda a partir de un valor base, el precio, las promociones, los fines de semana, ruido aleatorio y un factor de inflación. Luego separa el 20 % de los datos al azar, entrena un único RandomForestRegressor con hiperparámetros fijos y registra en MLflow sus parámetros, métricas y modelo. Al revisarlo, se nota que algunas variables no influyen en la demanda, que el efecto de cosecha se anula a cero, que los datos cambian cada día por usar datetime.now() y que la partición aleatoria filtra información del futuro.

Mejoras propuestas

Se propuso mejorar tanto los datos como el proceso de modelado:

- *Datos más realistas y reproducibles*: fecha inicial y semilla fijas, efecto de cosecha corregido con dos picos al año, y demanda que ahora sí depende de la temperatura, la lluvia y los feriados, con efecto logarítmico del precio y sin bfill().
- *Evaluación sin fuga de información*: partición cronológica 80/20 (entrenar con el pasado y evaluar con el futuro) y validación cruzada con TimeSeriesSplit.
- *Mejor modelado y trazabilidad*: modelar la demanda real deflactada por la inflación, y comparar Ridge, Random Forest y Gradient Boosting como corridas anidadas en MLflow, con la gráfica de predicción vs real como artefacto.



taller3_mlflow_v2.ipynb

C: > Users > marcs > Downloads > taller3_mlflow_v2.ipynb > Machine Learning Engineering - Taller 3

Generar | Código | Markdown | Ejecutar todo | Borrar todas las salidas | Esquema

```
print(tabla)

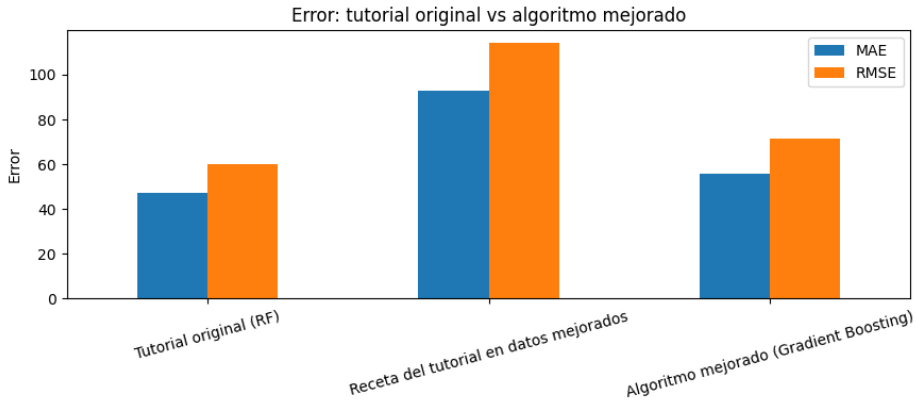
mejora = (1 - tabla.loc["Algoritmo mejorado (Gradient Boosting)", "RMSE"] / tabla.loc["Receta del tutorial en datos mejorados", "RMSE"]) * 100
print(f"\nReducción del RMSE (mismos datos): {mejora:.1f}%")

tabla[["MAE", "RMSE"]].plot(kind="bar", figsize=(9, 4), rot=15)
plt.title("Error: tutorial original vs algoritmo mejorado")
plt.ylabel("Error")
plt.tight_layout()
plt.show()
```

[9] Python

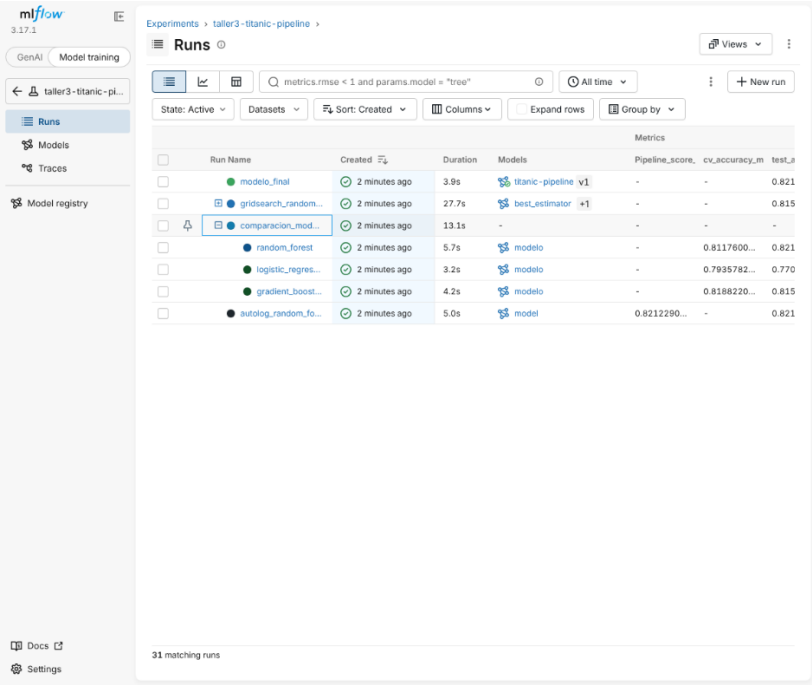
```
                                    MAE      RMSE      R2
Tutorial original (RF)              47.3746   60.1397  0.8853
Receta del tutorial en datos mejorados  92.9769  114.0483  0.7645
Algoritmo mejorado (Gradient Boosting)  55.6853   71.3598  0.9078

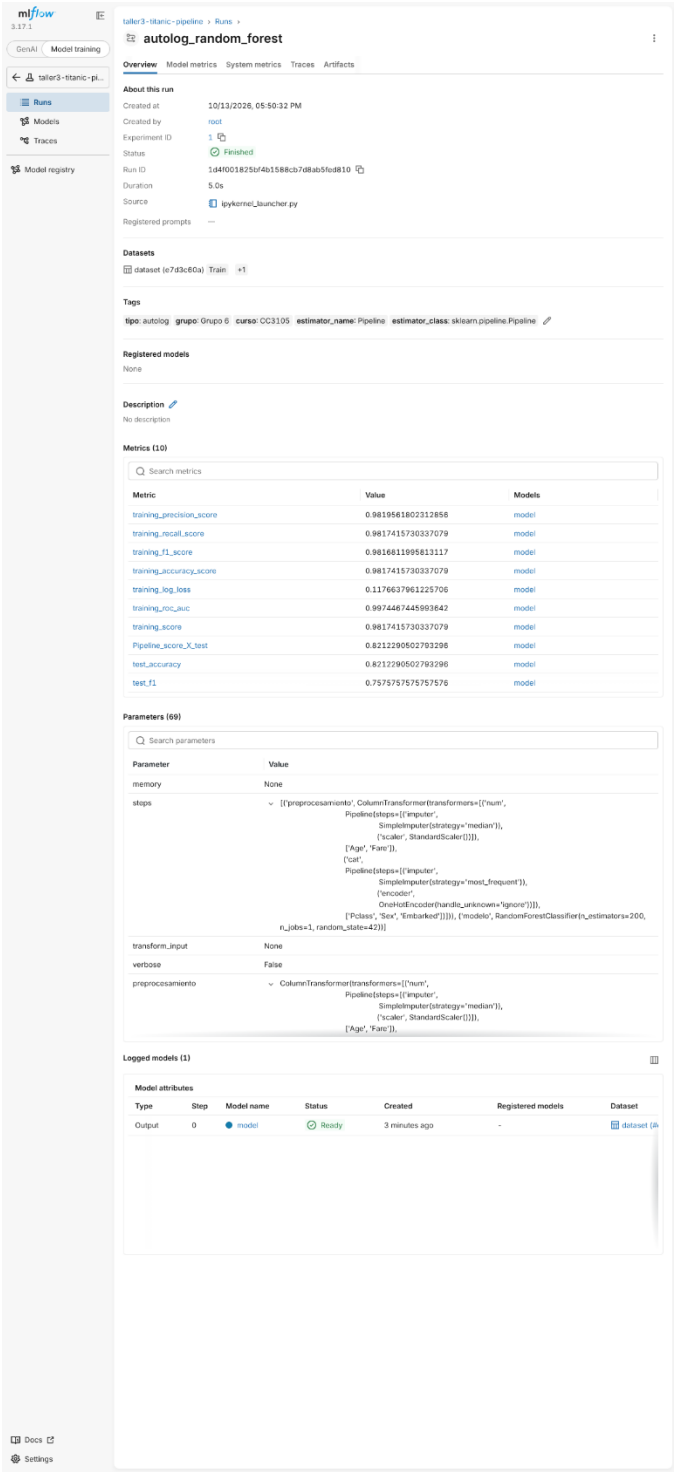
Reducción del RMSE (mismos datos): 37.4%
```

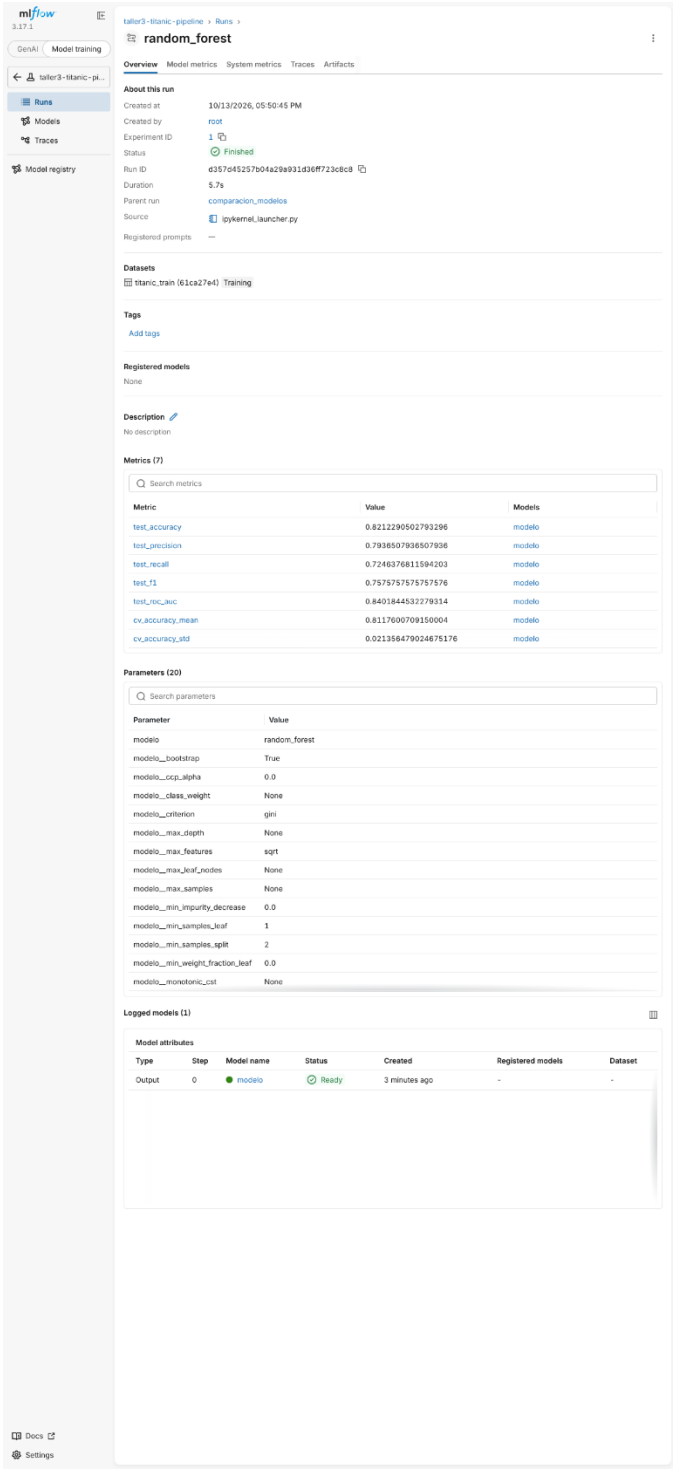


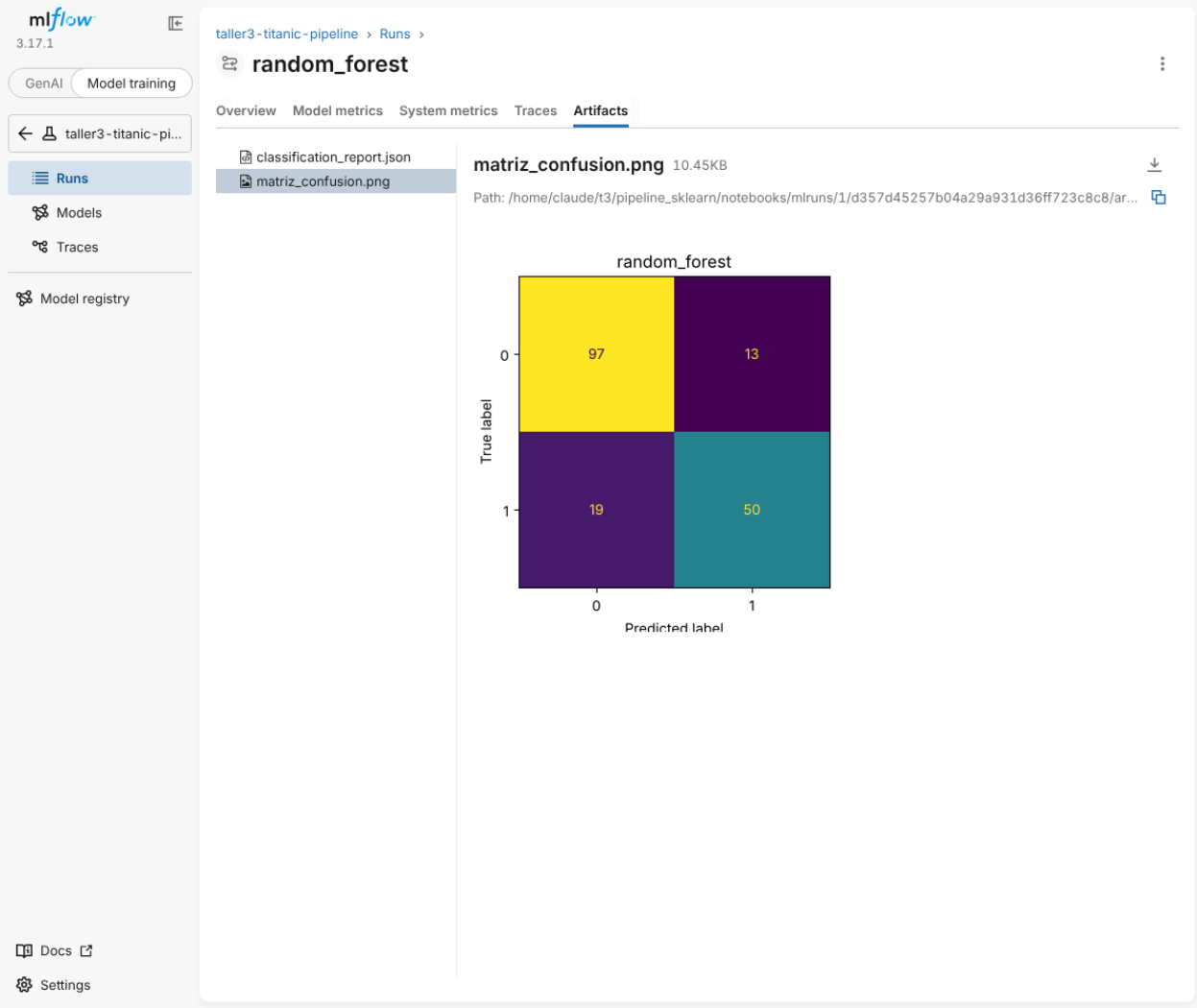
Con los cambios implementados, el algoritmo mejorado obtuvo mejores resultados que la receta original del tutorial al evaluarse sobre los mismos datos y el mismo conjunto de prueba: el RMSE bajó de 114.05 a **72.34**, el MAE de 92.98 a **56.88** y el R^2 subió de 0.765 a **0.905**. Esta mejora se debe a que los datos ahora reflejan mejor la realidad (la temperatura, la lluvia y los feriados sí influyen en la demanda y la estacionalidad de cosecha deja de anularse) y a que la evaluación cronológica elimina la fuga de información del futuro. Además, comparar varios algoritmos mostró que Gradient Boosting ajusta mejor que el Random Forest del tutorial, por lo que el nuevo enfoque resulta más preciso y confiable que el original.

3. MLFlow con pipeline de scikit-learn




mlflow
3.17.1
GenAI
Model training
taller3-titanic-pi...
Runs
Models
Traces
Model registry
Docs
Settings
taller3-titanic-pipeline › Runs ›
autolog_random_forest
Overview
Model metrics
System metrics
Traces
Artifacts
About this run
Created at 10/13/2026, 05:50:32 PM
Created by root
Experiment ID 1
Status Finished
Run ID 1d4f001825bf4b1588cb7d8ab5fed810
Duration 5.0s
Source ipykernel_launcher.py
Registered prompts —
Datasets
dataset (e7d3c60a) Train +1
Tags
tipo: autolog grupo: Grupo 6 curso: CC3105 estimator_name: Pipeline estimator_class: sklearn.pipeline.Pipeline
Registered models
None
Description
No description
Metrics (10)
Search metrics
Metric Value Models
training_precision_score 0.9819561802312656 model
training_recall_score 0.9817415730337079 model
training_f1_score 0.9816811995813117 model
training_accuracy_score 0.9817415730337079 model
training_log_loss 0.11766379612257 06 model
training_roc_auc 0.9974467445993642 model
training_score 0.9817415730337079 model
Pipeline_score_X_test 0.8212290502793296 model
test_accuracy 0.8212290502793296 model
test_f1 0.7575757575757576 model
Parameters (69)
Search parameters
Parameter Value
memory None
steps [('preprocesamiento', ColumnTransformer(transformers=[('num', Pipeline(steps=[('imputer', SimpleImputer(strategy='median')), ('scaler', StandardScaler())]), ['Age', 'Fare']), ('cat', Pipeline(steps=[('imputer', SimpleImputer(strategy='most_frequent')), ('encoder', OneHotEncoder(handle_unknown='ignore'))]), ['Pclass', 'Sex', 'Embarked'])])), ('modelo', RandomForestClassifier(n_estimators=200, n_jobs=1, random_state=42))]
transform_input None
verbose False
preprocesamiento ColumnTransformer(transformers=[('num', Pipeline(steps=[('imputer', SimpleImputer(strategy='median')), ('scaler', StandardScaler())]), ['Age', 'Fare']),
Logged models (1)
Model attributes
Type Step Model name Status Created Registered models Dataset
Output 0 model Ready 3 minutes ago - dataset (4


mlflow
3.17.1
GenAI
Model training
taller3-titanic-pi...
Runs
Models
Traces
Model registry
Docs
Settings
taller3-titanic-pipeline › Runs ›
random_forest
Overview
Model metrics
System metrics
Traces
Artifacts
About this run
Created at 10/13/2026, 05:50:45 PM
Created by root
Experiment ID 1
Status Finished
Run ID d357d45257b04a29a931d36ff723c8c8
Duration 5.7s
Parent run comparacion_modelos
Source ipykernel_launcher.py
Registered prompts —
Datasets
titanic_train (61ca27e4) Training
Tags
Add tags
Registered models
None
Description
No description
Metrics (7)
Search metrics
Metric Value Models
test_accuracy 0.8212290502793296 modelo
test_precision 0.7936507936507936 modelo
test_recall 0.7246376811594203 modelo
test_f1 0.7575757575757576 modelo
test_roc_auc 0.8401844532279314 modelo
cv_accuracy_mean 0.8117600709150004 modelo
cv_accuracy_std 0.021356479024675176 modelo
Parameters (20)
Search parameters
Parameter Value
modelo random_forest
modelo__bootstrap True
modelo__ccp_alpha 0.0
modelo__class_weight None
modelo__criterion gini
modelo__max_depth None
modelo__max_features sqrt
modelo__max_leaf_nodes None
modelo__max_samples None
modelo__min_impurity_decrease 0.0
modelo__min_samples_leaf 1
modelo__min_samples_split 2
modelo__min_weight_fraction_leaf 0.0
modelo__monotonic_cst None
Logged models (1)
Model attributes
Type Step Model name Status Created Registered models Dataset
Output 0 modelo Ready 3 minutes ago - -



El algoritmo y las pruebas corrieron satisfactoriamente.

Conclusiones

1. *Detección de fraude en un banco*: en casos con datos muy desbalanceados MLflow permitiría comparar modelos y umbrales de decisión (recall, precisión) y guardar el que mejor detecta fraudes sin bloquear clientes legítimos.
2. *Predicción de demanda en un negocio (inventarios o ventas):* al reentrenar el modelo cada mes con datos nuevos, MLflow registraría cada versión y su error, y permitiría volver a la anterior si la nueva empeora.
3. *Modelo de salud o de riesgo que debe auditarse:* el Model Registry deja constancia de qué modelo, con qué datos y parámetros, tomó cada decisión, lo que facilita reproducir resultados ante una revisión.

Referencias

MLflow. (2026). *ML Model Registry*. MLflow Documentation. https://mlflow.org/docs/latest/ml/model-registry/

MLflow. (2026). *Model Signatures and Input Examples*. https://mlflow.org/docs/latest/ml/model/signatures/

MLflow. (2026). *Model Evaluation*. MLflow Documentation. https://mlflow.org/docs/latest/ml/evaluation/